



cf-python vs. xarray

Reviewing and comparing our support (with
view to get xarray users to use cf-python)

Sadie Bartholomew
CMS Weekly meeting, 2026-06-12

Part 1 of 3

Background

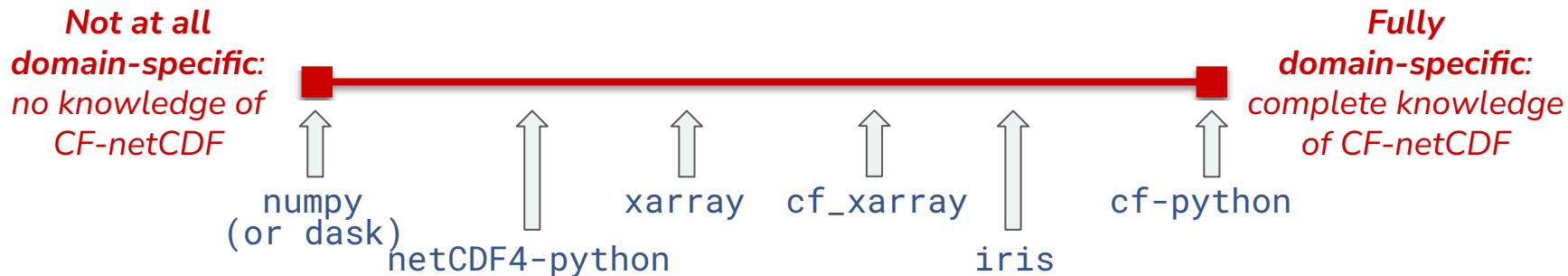


**National Centre for
Atmospheric Science**

NATURAL ENVIRONMENT RESEARCH COUNCIL

cf-python and related Python tools

Various Python tools you can use (not an exhaustive list!) to work with data in the earth sciences, including netCDF as one core format:

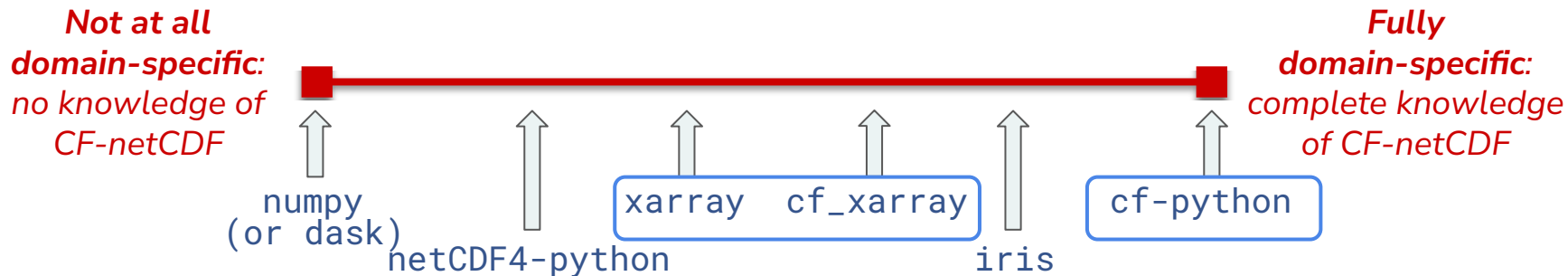


- `numpy`: (for completeness and diagram reference) the fundamental package for scientific computing in Python, supporting data arrays and operations on these only. Dask makes this efficient and lazy (and more)
- `netcdf4-python`: a tool for working with netCDF without CF Conventions awareness
- `(cf_)xarray`: `xarray` is a general tool which extends `numpy` to notably introduce labelling for dimensions, coordinates and attributes; `cf_xarray` adds limited CF Conventions awareness to this
- `iris`: Met Office product with similar functionality to `cf-python` but less CF support
- `cf-python`: NCAS product built on a complete implementation of the official CF data model



cf-python and related Python tools

Various Python tools you can use (not an exhaustive list!) to work with data in the earth sciences, including netCDF as one core format:



- `numpy`: (for completeness and diagram reference) the fundamental package for scientific computing in Python, supporting data arrays and operations on these only. Dask makes this efficient and lazy (and more)
- `netcdf4-python`: a tool for working with netCDF without CF Conventions awareness
- `(cf_)xarray`: `xarray` is a general tool which extends `numpy` to notably introduce labelling for dimensions, coordinates and attributes; `cf_xarray` adds limited CF Conventions awareness to this
- `iris`: Met Office product with similar functionality to `cf-python` but less CF support
- `cf-python`: NCAS product built on a complete implementation of the official CF data model



Why is it important to compare with xarray?

- Researchers and RSEs etc. very often use xarray to work with CF-compliant data (or data that *could* be CF compliant but maybe isn't or *isn't as much as it should be*)!
- xarray is one of the 'big boys' with a large dev team (<https://xarray.dev/team>) vs. our team of effectively two
- But xarray doesn't handle CF metadata as a *first class citizen* (read: *fully and in a robust way*)
- We'd like them to use cf-python, because we think it will help them to work with data that is somewhat or fully CF-compliant and preserve (or improve!) compliance
- Two cases to consider:
 1. They don't know about cf-python as an alternative
 2. They know about cf-python but prefer to use xarray for one or more reasons. We'd like to know those reason(s) so we can improve.



Part 2 of 3

Comparing cf-python to xarray support-wise



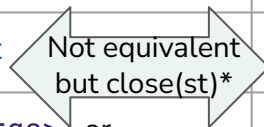
**National Centre for
Atmospheric Science**

NATURAL ENVIRONMENT RESEARCH COUNCIL

Basic API mapping between cf-python and xarray

- Both libraries offer Dask-backed lazy data array operations with metadata support
- Mapping between APIs for core supported functionality. Note the terminology differs:

<u>Object or functionality</u>	<u>In (cf_)xarray (import xarray as xr)</u>	<u>In cf-python (import cf)</u>
Data variable & all associated metadata*	<code>dataArray</code> as part of <code>Dataset</code>	<code>Field(s)</code> in a <code>FieldList</code>
Opening data	<code>ds = xr.open_dataset(<args>)</code> or <code>da = xr.open_dataarray(<args>)</code>	<code>f = cf.read(<args>)</code>
Data array access	<code>ds.data</code> or <code>ds.values</code>	<code>f.data</code> or <code>f.data.array</code>
Metadata e.g. coordinates and attributes access (units example)	<code>da.coords</code> <code>da.attrs</code> <code>da.attrs["units"]</code>	<code>f.dimension_coordinates()</code> & <code>f.auxiliary_coordinates()</code> <code>f.properties()</code> <code>f.get_property("units")</code>
Subspacing and collapsing	<code>da.sel(<args>)</code> Individual methods e.g. <code>da.mean(<args>)</code>	<code>f.subspace(<args>)</code> <code>f.collapse(<args>)</code>



*Not directly comparable since `cf.Fields` are fully CF aware and abide by a (and not just a but *the official*) CF data model, whereas `xr.DataArrays` are not as complete and structured for metadata



What cf-python offers that xarray does not

- **Full CF awareness** since is built upon the CF Data Model (built upon cfdm library)
- In cf-python we have sophisticated configurable **aggregation and reconstruction** abiding by well-defined rules (see https://ncas-cms.github.io/cf-python/aggregation_rules.html) but 'aggregation' in xarray is unformalised and piecemeal, via functionality such as multi-file reading (`xr.open_mfdataset("*.nc")`), concatenation (`xr.concat`) and 'merging' (`xr.merge`) and coordinate-based 'combination' (`xr.combine_by_coords`)
- cf-python has built-in **regridding**, but xarray has no native equivalent



What xarray offers that cf-python does not (besides a cooler name)

- **More intelligent and transparent data access/reading:** built-in support for opening datasets from many *engines* (via an engine kwarg e.g. `xr.open_dataset("*.nc", engine="netcdf4")`), with a backend architecture which seamlessly and flexibly handles e.g. cloud-native formats such as kerchunk, Zarr, parquet
- **Direct visualisation:** though we have cf-plot, this is a separate library and we can't call to plot as a method on a given object, whereas xarray supports plotting from the data object: `xr.DataArray.plot(<args>)`
- Better **interoperability** with the wider Python Data stack - tools are often built to integrate with, or build upon, xarray. The Pangeo stack is xarray centric for example.
- **Bells and whistles to make users' lives easier** e.g. User Experience sugar such as Notebook 'rich' formatting (see <https://github.com/NCAS-CMS/cf-python/issues/791>):

```
[5]: ds_weighted
```

```
[5]: xarray.Dataset
```

► Dimensions: (season: 4, y: 205, x: 275)

▼ Coordinates:

season	(season)	object	'DJF' 'JJA' 'MAM' 'SON'	 
xc	(y, x)	float64	189.2 189.4 189.6 ... 17.15 16.91	 
yc	(y, x)	float64	16.53 16.78 17.02 ... 27.76 27.51	 

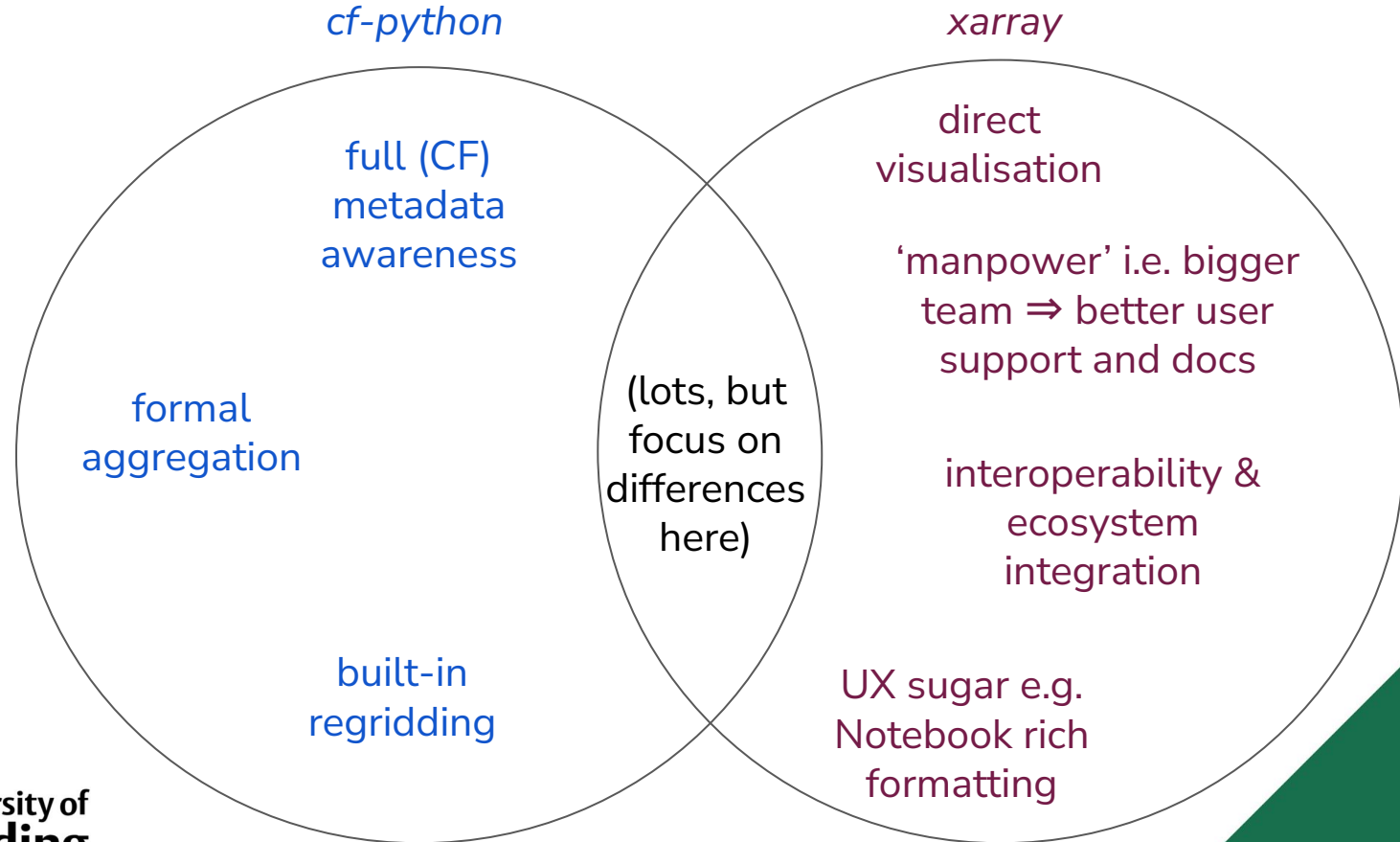
▼ Data variables:

Tair	(season, y, x)	float64	0.0 0.0 0.0 ... 22.08 21.73 21.96	 
------	----------------	---------	-----------------------------------	---

► Attributes: (13)



Summary of key differences (relating to CF/earth science only)



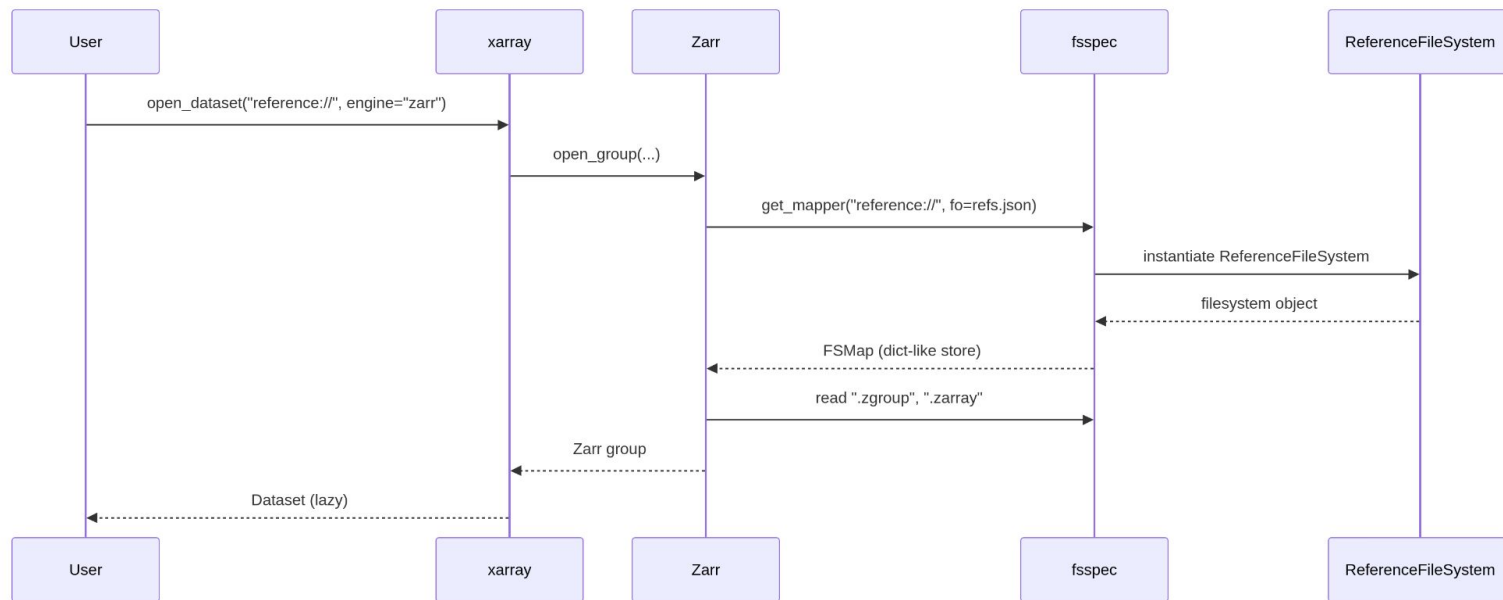
Comparing data access

- Both have sophisticated read capabilities, but the approach differs a bit. cf-python does much processing at read-time, whereas xarray tends to open and then manipulate afterwards.
- An `xr.open_mfdataset("*.nc")` will typically apply logic based on:
 - dimensions
 - coordinate values
 - merge/concat rules
- whereas a `cf.read("*.nc")` will:
 - discover multiple files
 - aggregate fragments
 - reason about CF identities
 - reconstruct fields



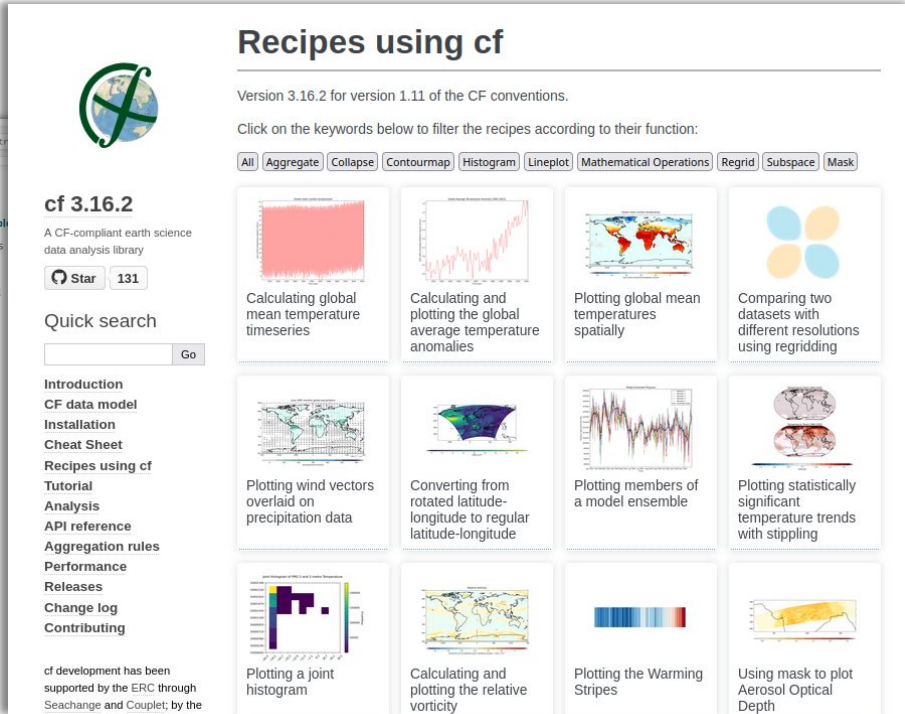
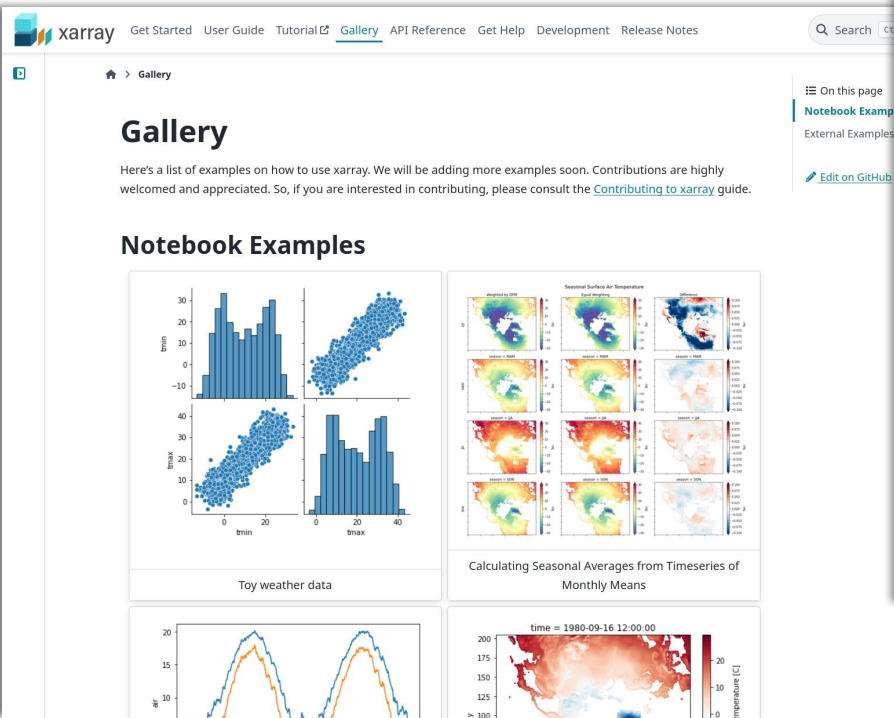
Comparing data access

- But the backend architecture is where xarray has a USP and is very powerful. xarray is designed to be the *common interface sitting above many storage systems*, whereas cf-python is designed to be the *common interface sitting above the CF data model*
- Zarr based data opening example, in xarray - though :



Recipes/examples to showcase usage

- We both provide 'recipe' examples in our docs showcasing illustrative usage:



Part 3 of 3

So, how can we poach xarray users?



**National Centre for
Atmospheric Science**

NATURAL ENVIRONMENT RESEARCH COUNCIL

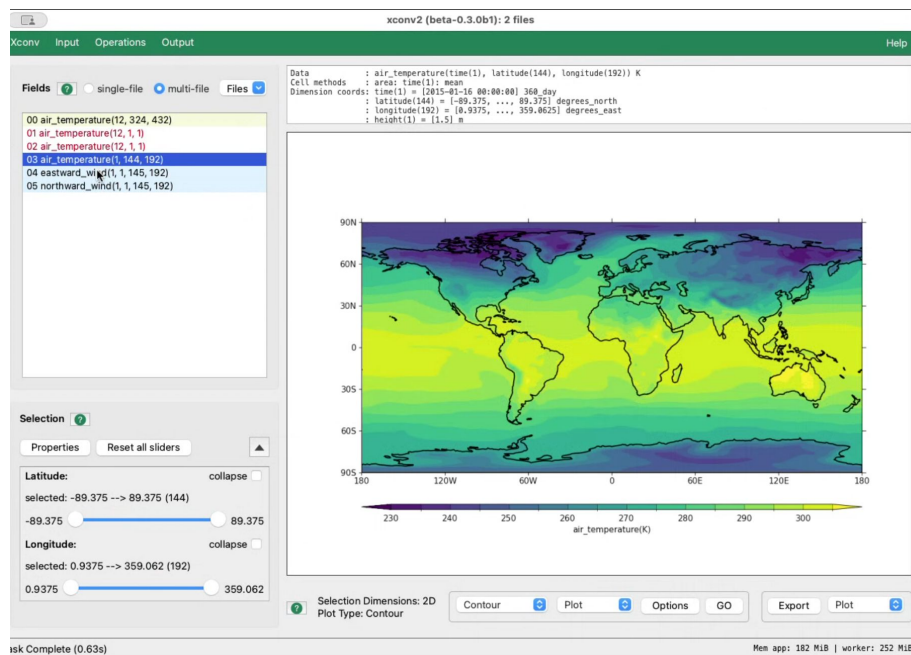
The easy wins

- We are working on improving our data access/reading support as part of EXPECT work and beyond
- Add a page in the docs to show how to translate/migrate code from xarray, including the table with the API mapping like my previous slide but more detailed and comprehensive as a 'cheat sheet' of sorts
- Could we even take some of the xarray recipes and show how to do the same in cf-python?
- Making users' lives easier generally with UX sugar etc. could be really helpful too



Broadcasting and advertising cf-python!

- xarray has a huge comm/developer team behind it and its incorporation into the Python Data ecosystem means it is very well-known and well-adopted
- But that doesn't mean we can't try to promote cf-python more!
- Downstream tools, notably xconv2, will help to showcase cf-python and its capabilities and draw in new users. More of this please!



Summary

- There are numerous tools (even just considering the Python-based ones) researchers can use to work with earth science data, with varying levels of domain specificity (notably here: knowledge of CF-netCDF)
- Our tool cf-python is fully domain-specific whereas (cf_)xarray is not, but the latter is more popular to use
- Key advantages of cf-python are: full CF metadata awareness, formal aggregation, built-in regridding
- Key advantages of xarray are (bearing in mind they have a much bigger dev team and backing!): smarter data reading/access, interoperability & ecosystem integration, direct visualisation, and interoperability & ecosystem integration
- We are working on getting cf-python's data reading/access in line with xarray's capabilities
- Core steps are to showcase how to translate from xarray to cf-python (e.g. a page or cheat sheet in the docs) and to generally broadcast cf-python more (widely)

